BÁO CÁO BR-04 — vLLM server trên Colab L4, ngân sách ngữ cảnh

1. Tóm tắt
2. Bảng nghiệm thu
3. Đối chiếu mười yêu cầu của BR-04
4. Đọc số
5. Phát hiện ngoài số đo
   5.1. Bẫy thứ tư: grammar quyết định không được thi hành — **đã sửa** (`82e9e9c`)
   5.2. Model chép thiếu dòng hàng vào tool — **MỞ, chờ quyết định**
   5.3. Ba script Colab không gửi `X-User-Id` — **đã sửa** (`bf63fef`)
   5.4. Model gọi thẳng, không tool, bịa số — ví dụ cho BR-05
6. Đề xuất quyết định — chờ chủ dự án chốt (SRS §8.5 câu 1)
7. Runbook — dựng lại cấu hình BR-04 trên Colab
8. Giới hạn của số đo

BÁO CÁO BR-04 — vLLM server trên Colab L4, ngân sách ngữ cảnh

Ngày đo: Thứ Hai 26/10/2026 · **Người làm:** Thành · **Đặc tả:** SRS_CHU_KY_01.md §BR-04 **Trạng thái:** số đo xong; **quyết định ctx chờ chủ dự án chốt** (mục 6); một lỗi mới **đang mở, chờ quyết định** (mục 5.2)

Phần cứng	Colab Pro, NVIDIA L4 **23.034 MiB**
vLLM	**0.11.2**, trong khoảng ghim `>=0.10.2,<0.12`
Model text	`anser-v3-awq` (Drive `ANSER_AI_Logistics/`), phục vụ bằng `vllm serve`, tên `anser-v3`
Vision	`Qwen/Qwen2.5-VL-3B-Instruct` bf16, **nạp trong tiến trình Brain** (`ENV=GPU`)
Brain	`ENV=GPU` + `VLLM_BASE_URL` → text qua HTTP, vision trong tiến trình, đúng cấu hình BR-04
Công cụ đo	`offline_training/do_ngan_sach_ctx.py` @ `bf63fef`; kiểm lại sau bản sửa grammar @ `82e9e9c`
Chi phí	**0đ/tháng** tăng thêm — Colab Pro sẵn có (NFR-02, R3)

---

1. Tóm tắt

1. **ctx 32k vừa trên L4 cùng vision đang nạp.** Cả ba mức dựng được, VRAM đỉnh 21,1 GB / 22,5 GB. Không cần hạ ctx, không cần hạ `gpu_memory_utilization`.
2. **Bể KV cố định 36.176 token ở** `gpu_memory_utilization=0.55`, không đổi theo `max_model_len`. Quy ra **1,1 chuỗi 32k** chạy song song.
3. **Continuous batching chạy thật**: 4 request đồng thời chỉ chậm hơn 1 request khoảng 6%.
4. **p95 ≈ 8,6–9,2 giây, vượt ngưỡng 5.000ms** — nhưng đó là thời gian sinh ~217 token ở ~25 token/giây, không phải hàng đợi. Ngưỡng cần định nghĩa lại (mục 6).
5. **Brain gọi được vLLM thật qua** `VLLM_BASE_URL` — log vLLM ghi `POST /v1/chat/completions 200`. Nghiệm thu BR-03 trên Colab.
6. **Lộ ra bẫy hỏng-im-lặng thứ tư, đã sửa**: grammar quyết định của vòng agentic **không được thi hành** trên đường server (mục 5.1).
7. **Lộ ra một lỗi R1 còn mở**: model **chép thiếu dòng hàng** vào tool `vat`, Brain báo hai con số tiền sai mà trông như có nguồn (mục 5.2). Hai cách sửa A và B đã viết sẵn, **chờ chủ dự án chọn**.

---

2. Bảng nghiệm thu

max_model_len	gmu	dựng được	VRAM đỉnh (MiB)	GPU KV cache (token)	request 32k song song	vision (MiB)	guided_json	p95 (ms)
8192	0.55	có	21156	36176	1.1	7874	đạt	1× → 8632; 4× → 9208
16384	0.55	có	21156	36176	1.1	7874	đạt	1× → 8650; 4× → 9159
32768	0.55	có	21144	36176	1.1	7874	đạt	1× → 8652; 4× → 9180

Cách đọc từng cột:

- **VRAM đỉnh** — lấy mẫu `nvidia-smi` mỗi giây suốt cả mức đo, vLLM + Brain + vision cùng nạp.
- **GPU KV cache** — dòng `GPU KV cache size` vLLM in lúc khởi động.
- **request 32k song song** — KV cache / 32.768.
- **vision** — VRAM tăng thêm khi Brain nạp runtime. Gồm cả CUDA context của tiến trình Brain, nên là cận trên của riêng model vision. Khớp số đo 7.876 MiB ngày 23/08.
- **guided_json** — gửi lược đồ `enum` + `additionalProperties: false` cùng một câu nhắc không liên quan ("kể chuyện vui"); đạt khi đầu ra vẫn đúng lược đồ. Chứng minh vLLM **thi hành** `guided_json`, không chỉ nhận.
- **p95** — xem định nghĩa ở mục 4.

Đo lại riêng mức 8192 sau bản sửa grammar (`82e9e9c`): KV cache, VRAM đỉnh, p95 (8.642 / 9.228 ms) không đổi.

---

3. Đối chiếu mười yêu cầu của BR-04

#	Yêu cầu	Trạng thái
1	`vllm serve` Qwen3-8B AWQ trên L4, trong khoảng ghim	**Đạt** — 0.11.2. Script chặn ngay nếu lệch khoảng ghim
2	Brain gọi qua `VLLM_BASE_URL`, trả lời **đúng** một câu thật đi hết vòng agentic	**Một phần.** Gọi được, đi hết vòng (2 lượt model, 1 tool). Nhưng câu trả lời có **số tiền sai** — mục 5.2
3	Đo **với vision đang nạp**	**Đạt** — 7.874 MiB ở mọi mức; script cảnh báo nếu dưới 4.000 MiB
4	Ba mức 8192 / 16384 / 32768: dựng được, VRAM đỉnh, KV cache	**Đạt** — mục 2
5	Quy KV ra số request 32k song song	**Đạt** — 1,1

#	Yêu cầu	Trạng thái
6	p95 ở mức đồng thời của yêu cầu 5	**Đã đo** ở 1× và 4× (NFR-03). **Vượt 5.000ms** — mục 4
7	32k không vừa thì hạ `gmu` trước rồi mới hạ ctx	**Không cần** — 32k vừa
8	ctx đủ cho kế hoạch 3 tool + RAG, **đo bằng ca thật**	**Chưa đo.** Câu thật duy nhất của phiên này đi 1 tool, không RAG
9	Quyết định vào ARCHITECTURE + sửa bảng §5.1	**Đã ghi dạng đề xuất** — chờ chủ dự án chốt (mục 6). §5.1 đã sửa theo số đo
10	Runbook dựng lại được	**Đạt** — mục 7

---

4. Đọc số

Vì sao KV cache không đổi theo `max_model_len`. Bể KV = phần `gpu_memory_utilization` giữ chỗ (0,55 × 23.034 ≈ 12,7 GB) trừ weights và activation. `max_model_len` chỉ là trần độ dài **một** request. Với paged attention, vLLM cấp KV theo số token request **thật sự dùng**, nên nâng `max_model_len` lên 32768 không làm request ngắn chạy kém song song đi. Chỉ khi một request **thật sự dài** 32k thì nó chiếm gần trọn bể.

Ước lượng ở SRS §2.3 (~0,147 MB/token) cho 36.176 token ≈ 5,3 GB KV — khớp bậc độ lớn với phần còn lại của 12,7 GB sau ~6 GB weights.

p95 đo gì. Độ trễ **một** lời gọi `/v1/chat/completions` tới vLLM, đo từ client, 20 lời gọi mỗi mức đồng thời. Câu nhắc là câu soát hoá đơn, `max_tokens=384`, `temperature=0.1`, không grammar. Model tự dừng (`finish_reason: stop`) sau **217 token** — không chạm trần.

Hệ quả: p95 ≈ 217 token × ~40ms ≈ 8,6 giây, tức **~25 token/giây** cho một request trên L4. p50 gần bằng p95 vì `temperature=0.1` cho đầu ra gần như y hệt mỗi lần. Ngưỡng 5.000ms ở ARCHITECTURE §13 chỉ đạt được với câu trả lời dưới ~125 token trên phần cứng này — cấu hình nào cũng vậy.

Batching. 1× → 4× chỉ thêm ~6% độ trễ, tức thông lượng tăng ~3,8 lần. Đây là thứ đường `vllm.LLM` trong tiến trình không có (ARCHITECTURE §5.4).

Thời gian một câu thật qua Brain. 87 giây cho câu hoá đơn sau bản sửa, **gồm cả lúc Brain nạp vision lần đầu**. Chưa tách được phần nạp khỏi phần sinh — lần đo sau nên hỏi hai câu liên tiếp.

---

5. Phát hiện ngoài số đo

5.1. Bẫy thứ tư: grammar quyết định không được thi hành — đã sửa (`82e9e9c`)

SRS §3.3 liệt kê ba bẫy hỏng-im-lặng của đường server. Đây là bẫy thứ tư.

Triệu chứng. Câu soát hoá đơn có kế hoạch `["vat"]`, nhưng `tool_calls=0` và `asked_back=False`, năm lần trên năm. Lối duy nhất trong vòng lặp ra được tổ hợp đó là nhánh `answer` — mà lược đồ bước-còn-kế-hoạch **không có trường** `answer`.

Nguyên nhân, đo bằng chính lược đồ của repo trên xgrammar (backend mặc định của vLLM 0.11):

Đầu ra thử	Phải	xgrammar (bản cũ)
gọi tool `vat` đúng	nhận	nhận
`{"thought", "answer"}` — trả lời tự do	**chặn**	**nhận**
`{"thought", "hoi_lai"}`	nhận	nhận
vừa gọi tool vừa hỏi lại	**chặn**	**nhận**

Lược đồ cũ đặt `properties` + `additionalProperties: false` ở gốc, cạnh `oneOf` + `not`. xgrammar dựng grammar từ các nhánh `oneOf` và **bỏ qua khoá đứng cạnh**. vLLM coi lược đồ là xgrammar hỗ trợ được (`has_xgrammar_unsupported_json_features → False`) nên không cảnh báo. `jsonschema` hiểu đúng, nên mọi test cũ vẫn xanh.

Sửa. Gốc chỉ còn `anyOf` (+ `$defs`); mỗi nhánh là object đóng đủ. Không `not`, không `oneOf`.

Bằng chứng sau sửa:

- `test_xgrammar_THAT_chan_quyet_dinh_nua_voi` chạy trên Colab với xgrammar thật: **passed** (cả hai ca `bat_buoc_tool`).
- Cả bộ trên Colab: **1041 passed, 4 skipped**.
- Câu hoá đơn qua Brain + vLLM thật: `tool_calls: 1`, log vLLM ghi **2** request của Brain (trước sửa: 1).

5.2. Model chép thiếu dòng hàng vào tool — MỞ, chờ quyết định

Sau bản sửa 5.1, tool `vat` có chạy, nhưng **đầu vào model điền bị thiếu**:

	Người dùng dán	Model gửi cho `vat`
Cước vận chuyển HN-HP	5.000.000	5.000.000
Phụ phí xăng dầu	500.000	**không có**

Tool tính đúng trên thứ nó nhận (5.000.000 + VAT 10% = 5.500.000), và Brain trả lời:

> *"...kết quả tính toán từ công cụ VAT cho thấy tổng hóa đơn thực tế là 5.500.000đ. Sự chênh lệch giữa tổng ghi và tổng thực tế là 1.000.000đ..."*

	Đúng	Brain nói
Tổng thực tế (VAT 10%)	**6.050.000**	5.500.000
Chênh lệch với 6.500.000	**450.000**	1.000.000
Kết luận "hoá đơn ghi sai"	đúng	đúng

Vì sao nghiêm trọng. Hai con số tiền sai, kèm câu "kết quả từ công cụ VAT" — sai mà **trông có nguồn**. Chốt neo số liệu (`guard_answer`) không bắt được vì cả hai số đúng là có trong kết quả tool. Docstring `build_decision_schema` đã cảnh báo đúng loại này: *"bịa đầu vào... tệ hơn vì con số bịa đi qua tool nên trông như*

có nguồn". Bản sửa 5.1 đóng lỗ "model không gọi tool"; lỗ này là "model gọi tool với đầu vào sai", và **chưa có chốt nào**.

Thêm: trường `thought` của bước gọi tool ghi "*không cần gọi công cụ*" trong khi vẫn gọi. Không dùng `thought` làm bằng chứng cho bất cứ điều gì.

Hai cách sửa, chờ chủ dự án chọn. Cả hai nằm ngoài phạm vi SRS chu kỳ 01 (BR-01 ghi "không sửa tool khác"), nên cần quyết định trước khi làm. Chọn một, chọn cả hai, hoặc hoãn sang chu kỳ 02 — trong lúc chờ, câu soát hoá đơn qua Brain **vẫn có thể ra số tiền sai**.

Cả hai cách đặt cùng một chỗ: `provider` trong `_run_agentic` (`chat.py`), chạy **sau** khi model điền tham số và **trước** khi tool chạy. Chỗ đó đã có sẵn cho `report` (`_provide_tool_data` ghi đè dữ liệu hệ thống), và nó thấy được câu gốc của người dùng.

Cách A — chốt neo đầu vào

Ý. Model điền tham số xong thì mã tất định đối chiếu: **mọi số tiền xuất hiện trong câu người dùng phải có mặt** trong tham số model gửi (`items[].price × qty` hoặc `stated_total`). Thiếu số nào thì **không chạy tool**, trả câu hỏi lại nêu đích danh số bị bỏ sót, ví dụ "*Tôi chưa đưa được khoản 500.000 vào phép tính — đây có phải một dòng của hoá đơn không?*". Đi đường `data_missing` sẵn có, nên `asked_back` được ghi vào metrics.

Sửa ở đâu	Hàm mới `so_tien_trong_cau(cau) -> list[int]` trong `src/core/` (tất định, có unit test); `provider` của `_run_agentic` gọi nó cho các tool lấy số từ lời người dùng: `vat`, `quote`
Bắt được	chép thiếu dòng (ca 5.2), chép sai chữ số (5.000.000 → 500.000), bịa thêm dòng không có trong câu (chiều ngược: số trong tham số không có trong câu)
Không bắt được	model điền đủ số nhưng gán sai vai (đổi `price` với `stated_total`); câu viết bằng chữ ("năm triệu rưỡi") nếu bộ đọc số chưa hiểu
Rủi ro báo nhầm	số không phải tiền trong câu: ngày (26/10/2026), số lượng, mã số thuế, biển số. Phải lọc: chỉ coi là tiền khi ≥ 4 chữ số hoặc có đơn vị (`đ`, `VND`, `nghìn`, `triệu`, `tr`), loại chuỗi dạng ngày và mã 10/13 số
Nghiệm thu	(1) model giả gửi 1 trên 2 dòng → không có `calculated_total` trong câu trả lời, câu hỏi lại nêu `500.000`; (2) model giả gửi đủ → tool chạy như cũ; (3) câu có ngày tháng và MST → không báo nhầm; (4) câu hoá đơn qua Brain + vLLM thật trên Colab → hoặc `tool_calls=1` với đủ 2 dòng, hoặc hỏi lại đúng số bị thiếu
Công	~1 ngày, phần lớn là bộ đọc số tiền tiếng Việt và các ca báo nhầm
Phủ	mọi cách người dùng viết hoá đơn, kể cả câu tự do — vì nó không cần hiểu cấu trúc, chỉ đối chiếu số

Cách B — tách hoá đơn bằng mã

Ý. Khi người dùng **dán hoá đơn có cấu trúc**, mã tự bóc thành `items` rồi **ghi đè** `items` model điền — đúng cách `report` đang làm với dữ liệu hệ thống, và đúng SO_TAY §9 mục 12: "*không để model điền tham số mà hệ thống có nguồn thật*". Model chỉ còn điền `stated_total` (và `default_is_reduced` nếu câu nói rõ).

Sửa ở đâu	Hàm mới `tach_hoa_don(cau) -> list[VatItem] | None` trong `src/core/`, nhận hai dạng: object JSON `{"tên": số}` (đúng dạng ca 5.2) và các dòng `tên: số` / `tên – số`. Không nhận dạng nào thì trả `None` và **giữ nguyên** `items` của model. `provider` của `_run_agentic` gọi nó cho `vat`, ghi log khi ghi đè
Bắt được	ca 5.2 và mọi hoá đơn dán đúng hai dạng trên — triệt để, vì model không còn chạm vào dòng hàng

Không bắt được	câu viết tự do ("cước 5 triệu, phụ phí năm trăm nghìn, tổng 6 triệu rưỡi") — vẫn do model điền, vẫn có thể chép thiếu
Rủi ro	bóc nhầm một dòng không phải hàng (ví dụ dòng "Tổng cộng: 6.500.000" bị coi là một dòng hàng). Phải loại các dòng có nhãn tổng / thuế / chiết khấu
Nghiệm thu	(1) JSON `{"Cước…": 5000000, "Phụ phí…": 500000}` → đúng 2 `VatItem`, `qty=1`; (2) dạng dòng `tên: 5.000.000đ`; (3) dòng "Tổng cộng" không thành dòng hàng; (4) model giả gửi 1 dòng → tool nhận 2 dòng; (5) không nhận dạng được → `items` của model giữ nguyên; (6) câu hoá đơn qua Brain + vLLM thật → `calculated_total = 6.050.000`, chênh 450.000
Công	~1 ngày
Phủ	chỉ hoá đơn dán có cấu trúc — nhưng đó là cách người dùng thật hay làm nhất (dán từ Excel, từ tin nhắn)

So hai cách

	A — chốt neo đầu vào	B — tách bằng mã
Bản chất	**phát hiện** model chép sai, rồi hỏi lại	**không cho** model chép, mã chép thay
Câu viết tự do	phủ	không phủ
Hoá đơn dán có cấu trúc	phủ, nhưng phải hỏi lại một lượt	phủ, trả lời đúng ngay
Rủi ro chính	báo nhầm số không phải tiền	bóc nhầm dòng tổng thành dòng hàng
Kết hợp	**bổ sung nhau**: B lo ca phổ biến cho đúng ngay; A là lưới cho mọi ca còn lại, kể cả khi B trả `None`	

Một câu hỏi chung, cả A và B đều chưa trả lời: số tiền người dùng dán là giá **trước thuế** hay **đã gồm VAT**? Tool `vat` luôn hiểu là trước thuế rồi cộng VAT. Nếu "Cước vận chuyển 5.000.000" trên hoá đơn thật đã gồm VAT, thì kết quả "6.050.000" cũng sai. Đây là câu hỏi nghiệp vụ, cần người hiểu cách Hoàng Phát ghi hoá đơn trả lời, không sửa được bằng mã.

5.3. Ba script Colab không gửi `X-User-Id` — đã sửa (`bf63fef`)

`GET /api/v1/task/{id}` chỉ trả task cho chủ (so `X-User-Id` với `user_id` của `POST /chat`), từ 15/08. `do_ngan_sach_ctx.py`, `chay_thu_live.py` và `bat_lai_brain.py` không gửi header này: hai script đầu chờ tới hết giờ trong khi câu trả lời đã nằm sẵn, script thứ ba ném `HTTPError` lúc hâm nóng (và thiếu cả token).

5.4. Model gọi thẳng, không tool, bịa số — ví dụ cho BR-05

Khi đo `finish_reason`, cùng câu hoá đơn gửi thẳng cho model (không vòng agentic, không tool) cho ra:

> *"Cước vận chuyển Hà Nội - Hải Phòng: 50.000.000 đồng … Tổng = 50 tr + 5 tr = 55 triệu … kết quả hiện tại chưa phù hợp và cần điều chỉnh để đạt tới mức 55 triệu đồng."*

Đọc 5.000.000 thành 50 triệu, giọng rất chắc. Brain không đi đường này (grammar khoá lối trả lời tự do khi có kế hoạch), nhưng đây là ví dụ nguyên văn cho loại **bịa số** trong bảng phân loại lỗi BR-05.

---

6. Đề xuất quyết định — chờ chủ dự án chốt (SRS §8.5 câu 1)

Tham số	Đề xuất	Lý do
`TEXT_MAX_MODEL_LEN`	**32768**	Đạt ctx 32k của S2; dựng được cùng vision; không làm request ngắn chạy kém song song (mục 4)
`gpu_memory_utilization`	**0.55**	Đủ chỗ cho 32k + vision, dư ~1,9 GB. Nâng lên 0.60 thêm ~7.800 token KV nhưng còn dư <1 GB — chưa đo, không đề xuất
`ANSER_TEXT_MAX_CONCURRENT`	**4**	Đã đo ở 4: chỉ +6% độ trễ. Bốn request cùng dài trọn 32k thì vượt bể, vLLM tự xếp hàng — chậm, không sập
Ngưỡng `inference_latency_p95`	**cần định nghĩa lại**	Đề xuất tách hai chỉ số: (a) p95 một **bước quyết định JSON** của vòng agentic (ngắn, ≤ 5.000ms khả thi), (b) **token/giây** cho câu trả lời dài (hiện ~25/request). Giữ một ngưỡng 5.000ms cho mọi câu trả lời là không đạt được trên L4

Chưa trả lời được yêu cầu 8 (ctx đủ cho kế hoạch 3 tool + RAG đo bằng ca thật). Đề xuất 32768 dựa trên việc nó **dựng được** và **không tốn gì thêm**, chưa dựa trên một ca dài thật.

---

7. Runbook — dựng lại cấu hình BR-04 trên Colab

Runtime **GPU L4** (T4 16GB không đủ). Mỗi ô một bước; ô nào in `x` thì dừng và đọc dòng đó.

```
# 1. Mã nguồn. Bản clone mới (repo private thì chèn token đọc):
#    !git clone -b feat/workflow-format-and-deterministic-core https://github.com/PCBoiz/ANSER_AI.git /c
%cd /content/ANSER_AI
!git pull -q && git log --oneline -1
```

```
# 2. GPU, Drive, model
!nvidia-smi --query-gpu=name,memory.total --format=csv      # phải là NVIDIA L4
from google.colab import drive; drive.mount('/content/drive')
!ls /content/drive/MyDrive/ANSER_AI_Logistics/anser-v3-awq | head
```

```
# 3. Thư viện. vLLM PHẢI trong khoảng ghim (SRS BR-02 yêu cầu 5)
!grep -vE '^(vllm|torch)' requirements.txt > req_colab.txt
!pip install -q -r req_colab.txt -r requirements-dev.txt "vllm>=0.10.2,<0.12"
!pip show vllm | grep Version
```

```
# 4. Kiểm mã trước khi đốt GPU: phải xanh, và hai ca xgrammar PHẢI passed (không skipped)
!ENV=LOCAL python -m pytest -q -rs tests/test_agentic_plan.py -k "xgrammar or anyOf"
!ENV=LOCAL python -m pytest -q 2>&1 | tail -3
```

```
# 5. Đo. Lần đầu tự tải trước vision/embedder/reranker (~1 phút); lần sau thêm --khong-tai-truoc
!python -u offline_training/do_ngan_sach_ctx.py
#   chỉ một mức:     --muc 32768
#   hạ giữ chỗ:      --gmu 0.5
```

```
# 6. Lưu kết quả lên Drive — runtime ngắt là mất. Mỗi lần chạy GHI ĐÈ /content/br04_*
!D=/content/drive/MyDrive/ANSER_AI_Logistics/br04_$(date +%d%m_%H%M) && mkdir -p $D && cp -r /content/br
```

```
# 7. Brain đã đi đường nào ở câu thật
import json
for d in open("/content/br04_logs/ai_metrics.jsonl"):
    r = json.loads(d); print({k: r.get(k) for k in ("route", "tool_plan", "tool_calls", "asked_back")})
```

Bẫy đã gặp khi dựng, đừng gặp lại:

Triệu chứng	Nguyên nhân	Xử lý
`git pull` : *untracked working tree files would be overwritten… tests/__init__.py*	bản clone cũ có file `touch` tay	`rm tests/__init__.py` **một lần** rồi pull. Đừng để `rm` trong ô chạy lặp: nó xoá file thật của nhánh
`No module named 'tests.test_knowledge'`	`tests/__init__.py` bị xoá; Colab có gói `tests` khác trong `dist-packages`	`git checkout -- tests/__init__.py`
Câu `/chat` đầu chờ mãi, Brain không dùng CPU	client không gửi `X-User-Id` khi hỏi kết quả task → 404 mỗi lần	đã sửa trong cả ba script (`bf63fef`)
`ERROR: Pipe to stdout was broken` sau `pip show … | head`	`head` đóng pipe	vô hại

---

8. Giới hạn của số đo

- **Một máy, một buổi.** Colab L4, không phải máy sẽ chạy production (chưa chốt, SRS §8.5 câu 2). Điều khoản Colab cấm phục vụ traffic thật.
- **p95 trên một câu nhắc**, 20 lời gọi mỗi mức, câu trả lời ~217 token. Không phải phân phối câu hỏi thật.
- **Chưa có request dài.** Không ca nào chạm gần 32k token; con số "1,1 chuỗi 32k" là phép chia, chưa phải đo.
- **Chưa đo yêu cầu 8** (kế hoạch 3 tool + RAG).
- **Compose chưa ai chạy.** `ENV=SERVER` (vision tắt) đã có phép thử đơn vị, chưa chạy trên máy thật.

- **Một câu thật duy nhất qua Brain** (câu hoá đơn). Nó đủ để lộ 5.1 và 5.2; không đủ để nói gì về chất lượng chung — đó là việc của BR-05.